

FinAI DevOps Deployment Documentation

1. Project Overview

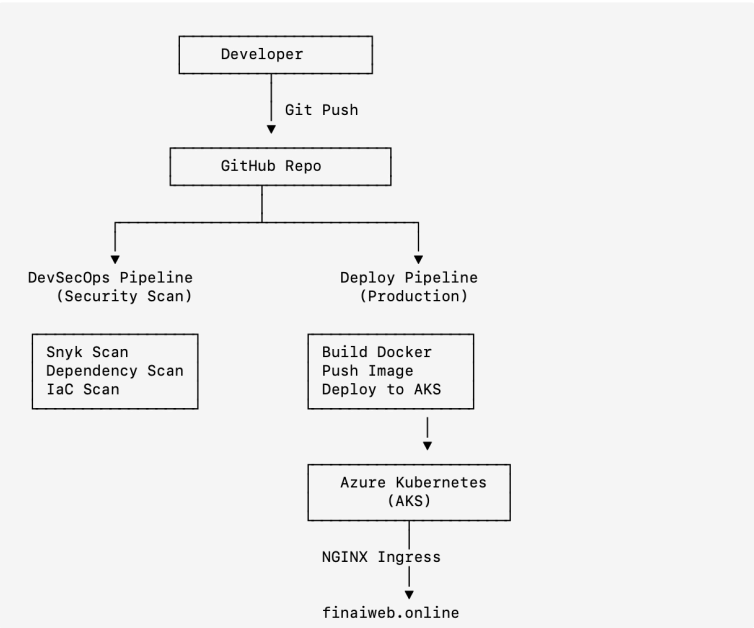
The goal of this project was to design and implement a **secure, automated, and scalable deployment pipeline** for the FinAI application using modern DevOps practices.

The system provisions infrastructure using **Terraform**, deploys the application on **Azure Kubernetes Service (AKS)**, and automates delivery through **GitHub Actions CI/CD pipelines** with integrated **DevSecOps security scanning**.

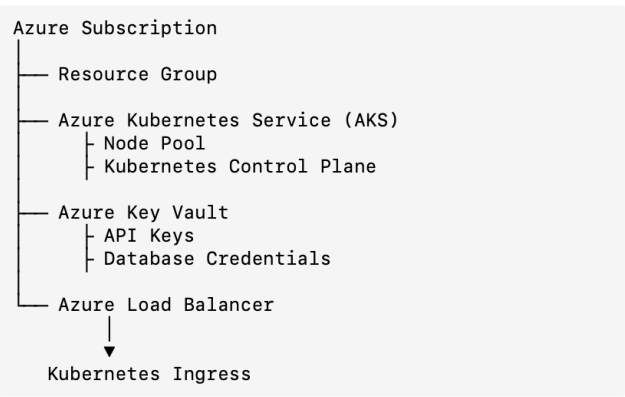
Additionally, the system integrates **domain routing, ingress networking, and monitoring** using **Prometheus and Grafana**.

2. System Architecture

High-level architecture:



3. Infrastructure Architecture



4. Infrastructure Provisioning (Terraform)

Terraform was used to provision Azure infrastructure in a reproducible and version-controlled manner.

Resources Provisioned

Resource Group
Azure Kubernetes Service (AKS)
Azure Key Vault
Networking Components
Logging Resources
Used GHCR instead of ACR to save cost

Terraform File Structure

```
infra/  
├── terraform/  
│   ├── provider.tf  
│   ├── resource-group.tf  
│   ├── keyvault.tf  
│   ├── container-app.tf  
│   ├── variables.tf  
│   ├── outputs.tf  
│   └── logs.tf
```

Terraform Workflow

Initialization: terraform init

Planning infrastructure: terraform plan

Applying infrastructure: terraform apply

Destroy infrastructure: terraform destroy

5. Application Containerization

The FinAI application is containerized using Docker.

Dockerfile: Docker image is built during the CI pipeline

Responsible for:

Installing dependencies
Building frontend
Running application server

6. Kubernetes Deployment

The application is deployed on AKS using Kubernetes manifests.

Kubernetes Components Used

Deployment: deployment.yml

Responsible for:

Running application pods
Maintaining replicas
Handling rolling updates
Self-healing

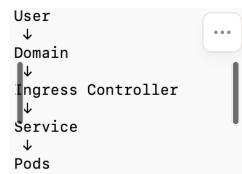
Service: **service.yml**

Type: Cluster IP

Provides internal networking between pods.

Ingress: **ingress.yml**

Routes external traffic to Kubernetes services.



7. CI/CD Pipelines

Two pipelines were implemented:

DevSecOps pipeline:

devsecops
succeeded 14 hours ago in 2m 11s

- > ✓ Set up job
- > ✓ Pull snyk/snyk:node
- > ✓ Build SonarSource/sonarqube-scan-action@v2
- > ✓ Build aquasecurity/trivy-action@0.24.0
- > ✓ Checkout repository
- > ✓ Setup Node
- > ✓ Install dependencies
- > ✓ SonarQube Scan
- > ✓ Snyk Scan
- > ✓ Login to GHCR
- > ✓ Build Docker Image
- > ✓ Trivy Scan
- > ✓ Push Image to GHCR
- > ✓ Post Login to GHCR
- > ✓ Post SonarQube Scan
- > ✓ Post Setup Node
- > ✓ Post Checkout repository
- > ✓ Complete job

Production deployment pipeline:

deploy

succeeded 4 days ago in 34s

- >  Set up job
- >  Checkout repository
- >  Azure Login
- >  Get AKS credentials
- >  Deploy to AKS
- >  Verify deployment
- >  Post Azure Login
- >  Post Checkout repository
- >  Complete job

8. DevSecOps Pipeline

The DevSecOps pipeline ensures security checks before deployment.

Security scans include:

Static Code Analysis

Detects insecure coding patterns.

Dependency Scanning

Checks for vulnerabilities in: package.json dependencies

Container Scanning

Scans Docker images for vulnerabilities.

Infrastructure Scanning

Scans Terraform configuration for misconfigurations.

Example checks:

Open security groups
Improper permissions
Exposed secrets

9. Production Deployment Pipeline

Triggered when code is merged into the main branch.

Pipeline stages:

Step 1: Install

dependencies: `npm install`

Step 2: Build application:

`npm run build`

Step 3: Build Docker image:

`docker build`

Step 4: Push Docker image: Push image to container registry.

Step 5: Deploy to Kubernetes:

`kubectl apply`

`kubectl rollout restart`

10. Branching Strategy

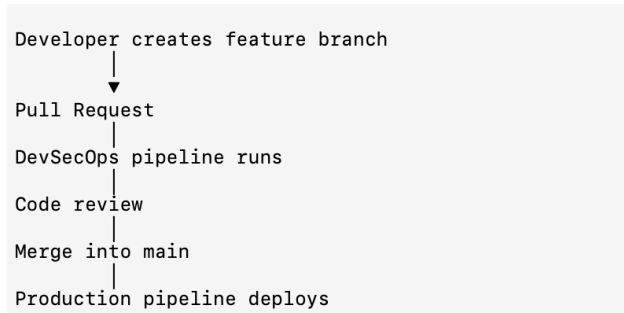
The project follows a controlled Git branching model.

Branches used:

main

feature/*

Workflow:



Benefits:

Prevents insecure code from reaching production

Ensures proper code review

Maintains stable production branch

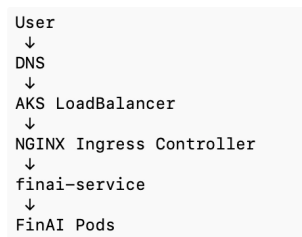
11. Domain and Traffic Routing

The application is exposed through a public domain.

Domain: finaiweb.online

DNS mapping: finaiweb.online → AKS LoadBalancer IP

Traffic flow:



12. HTTPS Security

HTTPS was enabled using Let's Encrypt certificates.

Components used:
cert-manager
ClusterIssuer
TLS secret

13. Monitoring Setup

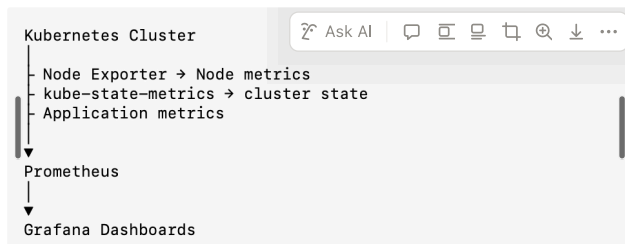
Monitoring stack includes:

Prometheus
Grafana
Alertmanager
Node Exporter
kube-state-metrics

Installed using Helm: kube-prometheus-stack
Monitoring capabilities:

CPU usage
Memory usage
Pod health
Node metrics
Cluster performance

Grafana dashboards visualize these metrics.



14. Security Practices Implemented

Security best practices used:

Infrastructure as Code
DevSecOps pipeline scanning
Secret management via Key Vault
Container vulnerability scanning
TLS encryption
Kubernetes self-healing

15. Scalability Features

The architecture supports scaling through:

Pod replicas
Load balancing
Kubernetes scheduling
Horizontal scaling

This ensures high availability and reliability.

16. Key Benefits of the Architecture

Fully automated deployments
Infrastructure reproducibility
Integrated security scanning
Scalable container orchestration
Centralized monitoring
Secure traffic routing

17. Conclusion

This project demonstrates a production-style DevOps workflow integrating infrastructure provisioning, automated CI/CD pipelines, security scanning, container orchestration, and monitoring.

The system enables secure and automated deployments from source code commit to a live production environment.

